

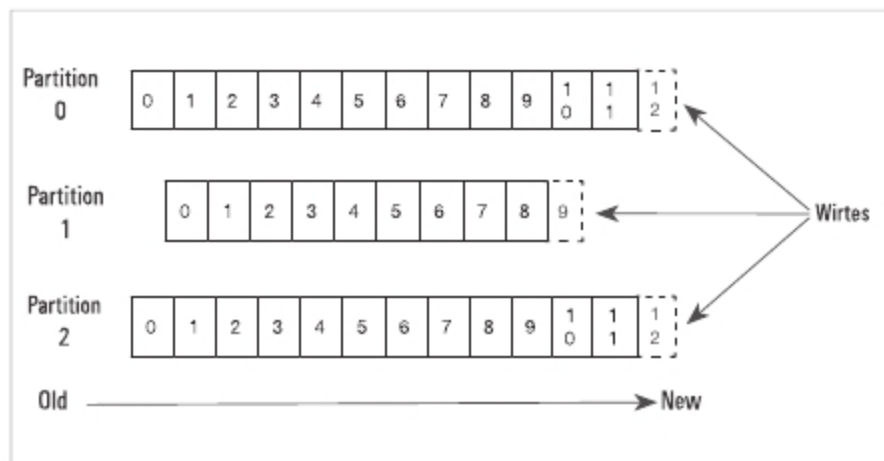


Figure 1: Anatomy of a topic

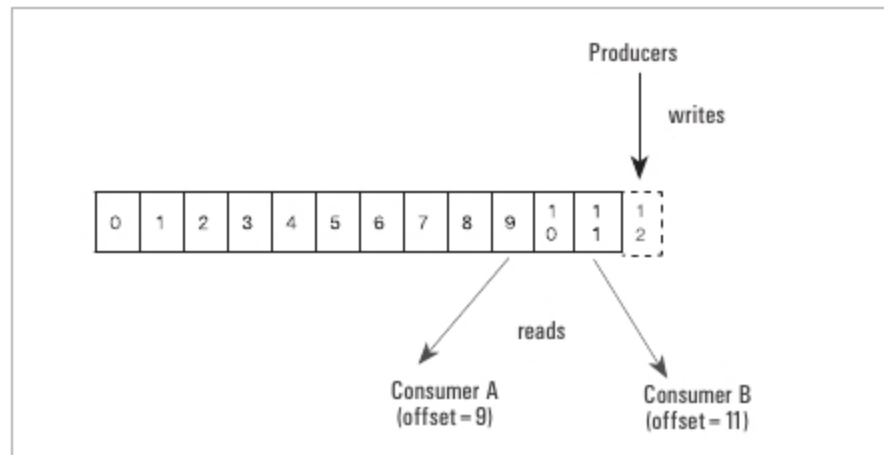


Figure 2: Structure of a partition

5. **Producers (an application):** Producers publish messages to the topics of their choice. The producer is responsible for choosing which record to assign to a particular partition within the topic.
6. **Consumers:** Consumers read data from brokers. They are applications that feed on data streams from topics in the Kafka cluster.
7. **Zookeeper:** This is used to manage and coordinate a Kafka broker. It is primarily used to notify producers and consumers about the presence of a new broker or about the failure of a broker in the Kafka system.

Zookeeper is a distributed configuration and synchronisation service, which serves as the coordination interface between the Kafka brokers and consumers. Since all the critical information is stored in Zookeeper and it normally replicates this data across its ensemble, failure of a Kafka broker or Zookeeper does not affect the state of the Kafka cluster.

The Kafka messaging workflow

Let us now focus on how Kafka works. The producer sends messages through Kafka topics which get partitioned and distributed across the cluster by brokers. Many producers and brokers share the workload to receive data. The consumer subscribes to the topic, and Kafka provides the current offset of the topic to the consumer and also saves the offset in the Zookeeper ensemble.

The consumer sends an acknowledgement to the Kafka broker. Once the acknowledgement is received, it changes the offset to a new value and updates Zookeeper. Consumers can join the consumer group. The number of partitions impacts

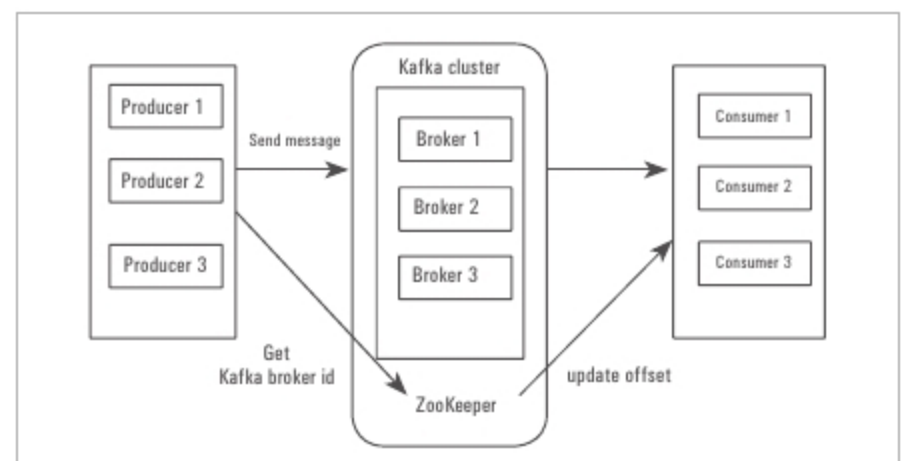


Figure 3: Apache Kafka architecture

the maximum parallelism of consumers, as we cannot have more consumers than partitions.

Kafka doesn't allow more than two consumers to read the same topic simultaneously. This restriction is required to avoid double reading of data. Moreover, Kafka guarantees that a message is only read by a single consumer in the group. Data/messages are never pushed out to the consumer. The consumer will ask for messages when it is ready to handle them. Consumers never overload themselves with lots of data or lose any data since all the messages are queued in Kafka.

You will need a Java Runtime Environment to install Kafka.

Step 1: Installing the Zookeeper framework

a) Downloading Zookeeper

To install the Zookeeper framework on your machine or standalone mode, download its latest version and extract the tar file using the following command:

```
http://zookeeper.apache.org/releases.html
tar -zxf zookeeper-3.4.6.tar.gz
cd zookeeper-3.4.6
mkdir data
```

b) Creating the configuration file

Open the configuration file named `conf/zoo_sample.cfg` and set the following parameters:

```
$ vi conf/zoo_sample.cfg
tickTime=2000
dataDir=/path/to/zookeeper/data
clientPort=2181
initLimit=5
syncLimit=2
```

Once the configuration file has been saved successfully, return to the terminal again and start the Zookeeper server.

c) Starting the Zookeeper server and getting connected

To do this, give the following command:

```
sudo bin/zkServer.sh start
```

After executing the above command, you will get the response shown below: